



UNIVERSIDADE EDUARDO MONDLANE
FACULDADE DE ENGENHARIA
DEPARTAMENTO DE ENGENHARIA ELECTROTÉCNICA

2º ano, Licenciatura em Engenharia Informática, Laboral

Cadeira: Linguagens de Programação

Estudo da linguagem de programação C++

Grupo - (02)

Discentes:

Boana, Álvaro Eduardo - 30%
Chiulele, Denzel Clauton Rogério- 40 %
Guambe, Állen Lénart Mateus - 50 %
Júnior, Raimundo Teixeira Dias- 30 %
Machuza, Edwin Bernardo - 15 %
Omar, Laiqa Rafik - 50 %

Docente:

Ivone Cipriano, Eng^a

Maputo, Abril de 2026



UNIVERSIDADE EDUARDO MONDLANE
FACULDADE DE ENGENHARIA
DEPARTAMENTO DE ENGENHARIA ELECTROTÉCNICA

2º ano, Licenciatura em Engenharia Informática, Laboral
Cadeira: Linguagens de Programação

Tema: Estudo da linguagem de programação C++

Membros do grupo:

Boana, Álvaro Eduardo
Chiulele, Denzel Clauton Rogério
Guambe, Allen Lénart Mateus
Júnior, Raimundo Teixeira Dias
Machuza, Edwin Bernardo
Omar, Laiqa Rafik

Maputo, Abril de 2026

Índice

1	Introdução	1
2	Objectivos	2
2.1	Objectivo geral	2
2.2	Objectivos específicos	2
3	Historial da linguagem C++	3
4	Programação Orientada à Objectos(POO)	4
4.1	Classe	4
4.2	Objecto	4
4.3	Abstração	4
4.4	Encapsulamento	4
4.5	Herança	5
4.6	Polimorfismo	5
4.6.1	Polimorfismo por herança	5
4.6.2	Polimorfismo por sobrecarga	5
5	Ficheiros em C++	6
6	Comparando C++ com outras linguagens	8
6.1	Velocidade de compilação	8
6.2	Paradigmas suportados	8
6.3	Segurança	8
6.4	Conclusão	8
7	Conclusão	9
	Referências	10

1 Introdução

A evolução da tecnologia e o desenvolvimento de sistemas computacionais cada vez mais complexos exigem o uso de metodologias e paradigmas que facilitem a organização, manutenção e reutilização do código. Nesse contexto, destaca-se a Programação Orientada a Objectos (POO), um paradigma que permite modelar problemas do mundo real através da utilização de classes e objectos, promovendo maior modularidade, flexibilidade e eficiência no desenvolvimento de software.

A linguagem de programação C++ surge como uma das principais ferramentas para a aplicação desse paradigma, uma vez que combina características de linguagens de baixo e alto nível, permitindo tanto o controlo detalhado do hardware quanto a implementação de estruturas abstratas e complexas. Além disso, o C++ é amplamente utilizado em diversas áreas, tais como desenvolvimento de sistemas, jogos, aplicações de alto desempenho e sistemas embarcados.

2 Objectivos

Na presente secção apresenta-se os objectivos a serem alcançados, nomeadamente, o geral em que é apresentado o objectivo principal do tema em estudo e específicos onde se estabelecem pontos que devem ser abordados para efectivação do objectivo geral.

2.1 Objectivo geral

Estudar a linguagem de programação C++ com enfoque no paradigma orientado a objectos.

2.2 Objectivos específicos

- Apresentar um breve historial da linguagem C++;
- Explicar os pilares do paradigma orientado à objecto e sua aplicação;
- Apresentar de forma explícita e directa ficheiros em C++.

3 Historial da linguagem C++

A história do C/C++ começa nos anos 70 com a invenção da linguagem C. A linguagem C foi inventada e implementada pela primeira vez por Dennis Ritchie em um DEC PDP-11, usando o sistema operacional UNIX.

A linguagem C é o resultado do processo de desenvolvimento iniciado com outra linguagem, chamada BCPL, desenvolvida por Martin Richards. Esta linguagem influenciou a linguagem inventada por Ken Thompson, chamado B, a qual levou ao desenvolvimento da linguagem C.

A linguagem C tornou-se uma das linguagens de programação mais usadas. Flexível, ainda que poderosa, a linguagem C tem sido utilizada na criação de alguns dos mais importantes produtos de software dos últimos anos. Entretanto, a linguagem encontra seus limites quando o tamanho de um projeto ultrapassa um certo ponto.

Ainda que este limite possa variar de projeto para projeto, quanto o tamanho de um programa se encontra entre 25.000 e 100.000 linhas, torna-se problemático o gerenciamento, tendo em vista que é difícil compreendê-lo como um todo. Para resolver este problema, em 1980, enquanto trabalhava nos laboratórios da Bell, em Murray Hill, New Jersey, Bjarne Stroustrup acrescentou várias extensões à linguagem C e chamou inicialmente esta nova linguagem de “C++ com classes”.

Entretanto, em 1983, o nome foi mudado para C++. Muitas adições foram feitas pós-Stroustrup para que a linguagem C pudesse suportar a programação orientada a objetos, às vezes referenciada como POO. Stroustrup declarou que algumas das características da orientação a objetos do C++ foram inspiradas por outra linguagem orientada a objetos chamada de Simula67.

A linguagem C é frequentemente referenciada como uma linguagem de nível médio, posicionando-se entre o assembler (baixo nível) e o Pascal (alto nível). Uma das razões da invenção da linguagem C++ foi dar ao programador uma linguagem de alto nível que poderia ser utilizada como uma substituta para a linguagem assembly.

4 Programação Orientada à Objectos(POO)

A Programação Orientada à Objectos é um paradigma da programação que se debruça sobre a representação de objectos do mundo real em código. O desenvolvimento de sistemas mais complexos implica maior número de linhas de código que torna difícil o controlo e actualização dos mesmos e reutilização do código para utilidades semelhantes.

E foi na ambição de resolver os problemas supracitados e outros que se desenvolveu a POO ou do inglês OOP - Programação Orientada à Objectos. Como o nome já diz orientada à **objectos**, é de elevada importância entender o conceito de objecto e como ele é criado, para tal serão definidos a **classe** e o **objecto**.

4.1 Classe

É um molde para criar objectos que irão possuir mesmas características(atributos) e comportamentos(métodos) que serão úteis para alterar e visualizar os atributos, é como uma forma de bolos circular(classe) que irá produzir bolos com o formato circular(objecto).

4.2 Objecto

Um objeto é uma instância de uma classe que contém atributos e métodos, permitindo representar entidades do mundo real e controlar o acesso aos seus dados.

A programação Orientada à Objectos é regida por quatro principais pilares: abstração, encapsulamento, herança e polimorfismo.

4.3 Abstração

Abstração é o processo de representar apenas as características relevantes de um objeto, ocultando os detalhes desnecessários (esconde a complexidade). Exemplo: O utilizador do ATM, ele só vê as instruções levantar, depositar, etc, ele não precisa saber o que acontece internamente para ele receber a informação final.

4.4 Encapsulamento

Encapsulamento é o mecanismo que restringe o acesso directo aos dados de um objeto, permitindo que eles sejam manipulados apenas por meio de métodos definidos.

4.5 Herança

Herança é o processo em que um objecto pode adquirir as propriedades de outro objecto. Isso é importante, já que suporta o conceito de classificação. Por exemplo, uma deliciosa maçã vermelha é parte da classificação maçã, que por sua vez, faz parte da classificação frutas, que está na grande classe comida.

Sem o uso de classificações, cada objeto precisaria definir explicitamente todas as suas características. Portanto, usando-se classificações, um objecto precisa definir somente aquelas qualidades que o tornam único dentro de sua classe. Ele pode herdar as qualidades que compartilha com a classe mais geral. É o mecanismo de herança que torna possível a um objecto ser uma instância específica de uma classe mais geral.

4.6 Polimorfismo

O polimorfismo é um conceito fundamental da Programação Orientada a Objetos (POO) que significa literalmente “muitas formas”. Em programação, ele permite que objectos de diferentes classes sejam tratados de forma uniforme, desde que compartilhem uma mesma interface ou herança. Existem dois tipos de polimorfismo: por herança e por sobrecarga(overloading).

4.6.1 Polimorfismo por herança

Se você tem uma classe base e várias classes derivadas, você pode usar uma referência da classe base para chamar métodos de qualquer classe derivada. O método correto será escolhido em tempo de execução.

4.6.2 Polimorfismo por sobrecarga

É quando você define vários métodos com o mesmo nome, mas com parâmetros diferentes na mesma classe. O compilador escolhe qual método usar dependendo dos argumentos(tempo de compilação).

5 Ficheiros em C++

Ficheiros servem para armazenamento persistente de dados, configuração de sistemas, registro de logs e intercâmbio de informações. Eles permitem guardar dados após o fim da execução do programa (memória não volátil), criando documentos legíveis por humanos e portáteis entre diferentes sistemas.

Dado que a informação que é recebida via teclado e é perdida após a execução do programa, pode ser do interesse do programador que as informações sejam guardadas em algum lugar, e por estes motivos que será apresentado abaixo um programa representativo.

Tal como visto em `Java` em `C++` também é possível manipular ficheiros de texto, abaixo será apresentada a forma de criação, escrita e leitura dos ficheiros.

Para efeitos educativos será apresentado um código que recebe o nome e curso e guarda num ficheiro de texto e de seguida mostra o conteúdo do mesmo.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    string nome, curso;
    cout << "Digite seu nome: ";
    getline(cin, nome);
    cout << "Digite seu curso: ";
    getline(cin, curso);

    // Abrir o ficheiro para escrita (cria se nao existir)
    ofstream ficheiroEscrita("dados.txt", ios::app);

    if (!ficheiroEscrita.is_open()) {
        cout << "Erro ao abrir o ficheiro para escrita!" << endl;
        return -1;
    }
```

```

//Escrever no ficheiro
ficheiroEscrita << "-----" << endl;
ficheiroEscrita << "Nome: " << nome << endl;
ficheiroEscrita << "Curso: " << curso << endl;
ficheiroEscrita << "-----" << endl;

ficheiroEscrita.close(); // fechar ficheiro

// Abrir o ficheiro para leitura
ifstream ficheiroLeitura("dados.txt");

if (!ficheiroLeitura.is_open()) {
    cout << "Erro ao abrir o ficheiro para leitura!" << endl;
    return -1;
}

// Ler linha por linha e mostrar o conteudo
string linha;
cout << "\nConteudo do ficheiro:\n";
while (getline(ficheiroLeitura, linha)) {
    cout << linha << endl;
}
ficheiroLeitura.close(); // fechar ficheiro
return 0;
}

```

⁰É considerada boa prática fechar o ficheiro, pois os dados muitas vezes ficam no buffer(memória) e só ao fechar o ficheiro o sistema garante que tudo foi escrito no disco.

6 Comparando C++ com outras linguagens

Comparar linguagens de programação(para achar a melhor) em tese é praticamente impossível, pois cada linguagem é desenvolvida com o propósito de resolver dado problema, contudo, podemos comparar em alguns pontos como velocidade de compilação, quantidade de paradigmas suportados, segurança, entre outros.

6.1 Velocidade de compilação

A linguagem C++ relativamente as outras linguagens quanto a velocidade de compilação dado uso intenso de headers(`#include`), falta de um sistema de modulos eficiente, linking pesado entre projectos, em projectos grandes pode levar muito tempo a compilar, a única linguagem actual que tende a levar tanto tempo ou mais quanto ao tempo de compilação é a Rust, devido ao sistema de tipos e verificações de segurança. Sendo GO a mais rápida.

6.2 Paradigmas suportados

A linguagem C++ é uma das poucas linguagens com um suporte aos paradigmas orientado a objecto, funcional, procedural, baixo nível, genérico, metaprogramação, entre outros. Que a torna uma das raras linguagens com suporte a tantos paradigmas.

6.3 Segurança

Quanto é segurança a linguagem C++ não é considerada muito segura devido a gerenciamento manual de memória, ponteiros perigosos, buffer overflow possível(quando um programa escreve mais dados, em um espaço de memória, que ele pode suportar), comportamento indefinido(é quando o programa faz algo que a linguagem não define o que deve acontecer).

6.4 Conclusão

Comparar linguagens de programação é, no fundo, avaliar compromissos (trade-offs) — nenhuma linguagem é “melhor em tudo”, como Python está para IA’s e análise de dados, C++ está para desenvolvimento de softwares e Kotlin e Swift para aplicações Mobile. A comparação no fim de contas depende de um contexto.

7 Conclusão

Tendo feito o trabalho, conclui-se que a linguagem C++, com o paradigma de Orientação a Objectos, consolida os conhecimentos dos pilares de POO e facilita a compreensão dos mesmos para a implementação destes em outras linguagens, uma vez que, a linguagem C++ foi uma das primeiras a implementar a POO de forma robusta em códigos e/ou programas.

Referências

- [1] Harvey M. Deitel e Paul J. Deitel. *Como Programar C++*. Pearson, 2006.
- [2] Brometheus. *c++ Full Course for free*. 2022. URL: <https://youtu.be/-Tko08Z07hI?si=VmXI05g30KkxwpZm>.
- [3] W3Schools. *C++ Files*. URL: https://www.w3schools.com/cpp/cpp_files.asp.